UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

PRIMER PARCIAL

MATERIA:

INGENIERÍA DE SOFTWARE I

GRUPO:

SB

DOCENTE:

ING. ROLANDO ANTONIO MARTINEZ CANEDO

ESTUDIANTE:

CHOQUE CANAVIDES CRISTIAN SANTIAGO 222087951

SANTA CRUZ-BOLIVIA

PERIODO 2/2026

UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

PRIMER PARCIAL

MATERIA:

INGENIERÍA DE SOFTWARE I

GRUPO:

SB

DOCENTE:

ING. ROLANDO ANTONIO MARTINEZ CANEDO

ESTUDIANTE:

CHOQUE CANAVIDES CRISTIAN SANTIAGO 222087951

SANTA CRUZ-BOLIVIA

PERIODO 2/2026

Contenido

1. PERFIL DEL PROYECTO.....1
1.1 Introducción..... 1
1.2 Antecedentes.....2
1.3 Planteamiento del problema.....2
1.4 Justificación..... 3
1.4.1 Justificación técnica.....3
1.4.2 Justificación académica.....3
1.4.3 Justificación de productividad y calidad.....3
1.4.4 Justificación de innovación..... 4
1.5 Objetivos.....4
1.5.1 Objetivo general.....4
1.5.2 Objetivos específicos.....4
1.7 Alcance..... 5
1.7.1 Alcance funcional.....5
1.7.2 Fuera de alcance de la primera versión..... 6
2. FUNDAMENTACIÓN TEÓRICA..... 7
2.1. Ingeniería de Software Asistida por Computadora (CASE).....7
2.1.1 Concepto.....7
2.1.2 Clasificación tradicional..... 7
2.1.3 Funciones relevantes.....7
2.1.4 Relación con el proyecto.....7
2.2 Desarrollo de Software Basado en Componentes.....8
2.2.1 Concepto de componente.....8
2.2.2 Principios aplicables..... 8
2.2.3 Aplicación al proyecto.....8
2.3 Arquitectura de Software..... 8
2.3.1 Concepto.....8
2.3.2 Atributos de calidad relevantes.....9
2.3.3 Arquitectura por capas.....9
2.3.4 Relación con el proyecto.....9
2.4 UML 2.5.1, OMG y arquitectura de cuatro niveles.....10
2.4.1 UML..... 10
2.4.2 Object Management Group.....10
2.4.3 Arquitectura de cuatro niveles de metamodelado..... 10
2.4.4 XMI e interoperabilidad..... 11
2.4.5 Aplicación en la herramienta.....11
2.5 Técnica de Modelado de Objetos (TMO/OMT).....11
2.5.1 Definición..... 11

2.5.2 Modelos de TMO....11
2.5.3 Fases generales.... 12
2.5.4 Relación con el proyecto....12
2.6 Inteligencia Artificial en el Desarrollo de Software....12
2.6.1 Aplicación en ingeniería de software.... 12
2.6.2 Modelos de lenguaje y lenguaje natural.... 12
2.6.3 Agentes de IA.... 13
2.6.4 Funciones de IA en el proyecto.... 13
2.6.5 Control y calidad de las operaciones inteligentes....13
2.7 Spring Boot.... 13
2.7.1 Concepto....13
2.7.2 Elementos relevantes para el generador....14
2.7.3 Estructura backend generada....14
2.7.4 Relación con el proyecto....14
3. PROCESO DE DESARROLLO....15
3.1 Flujo de trabajo: Captura de requisitos....15
3.1.1 Identificar actores y casos de uso.... 15
3.1.2 Priorización de casos de uso....16
3.1.3 Detallar casos de uso.... 16
3.1.4 Estructurar casos de uso....26
3.2.1 Análisis de paquetes....27
3.2.2 Análisis de casos de uso.... 30
3.2.3 Analizar clases....31
3.2.4 Análisis de paquete.... 32
3.3 Flujo de trabajo: Diseño....33
3.3.1 Diseño de arquitectura....33
3.3.1.1 Diseño de arquitectura física.... 33
3.3.1.2 Diseño de arquitectura Logica....34
3.3.2 Diseño de Datos.... 34
3.3.2.1 Diseño de datos logicos....34
3.3.2.2 Diseño de Datos Físicos....34
3.3.3 Diseño de Caso De Uso.... 34
3.3.3.1 Diagrama de secuencia.... 34
3.4 Flujo de trabajo: Implementación....34
3.4.1 Herramienta de Desarrollo Del Software.... 34
3.4.2 Implementacion de la arquitectura de sistema.... 34
3.4.3 Implementacion de la Arquitectura del Subsistema....34
3.5 Flujo de trabajo: Pruebas....34
4. RESULTADOS ESPERADOS....34
5. CONCLUSIONES....35
6. RECOMENDACIONES.... 35

7. BIBLIOGRAFÍA.....36
8. ANEXOS PREVISTOS.....37

1. PERFIL DEL PROYECTO

1.1 Introducción

El desarrollo de software requiere representar problemas complejos mediante modelos que permitan comprender la estructura de una solución antes de programarla. En proyectos donde varias personas participan en el análisis y diseño, el modelado también debe facilitar la coordinación, trazabilidad y reutilización de información. Las herramientas CASE apoyan estas actividades mediante editores gráficos, repositorios de modelos, validaciones y mecanismos de transformación de artefactos (Pressman & Maxim, 2020; Sommerville, 2016).

Este proyecto propone una herramienta colaborativa de ingeniería de software orientada al diseño conceptual de bases de datos mediante UML, con énfasis en diagramas de clases. El usuario podrá crear clases, atributos, operaciones y relaciones de manera manual, mientras la plataforma mantendrá una representación estructurada del modelo para permitir validación, colaboración, versionado, importación, exportación y generación posterior de software.

La propuesta incorpora inteligencia artificial en tres funciones concretas. Primero, interpretar instrucciones de texto o voz para ejecutar modificaciones controladas sobre el modelo. Segundo, analizar imágenes de diagramas UML y reconstruir una propuesta editable. Tercero, proporcionar un agente de asistencia que enseñe al usuario cómo usar la aplicación, explique conceptos UML y ofrezca orientación contextual según la tarea que está realizando.

A partir de un modelo validado, la herramienta generará la estructura inicial de un backend basado en Spring Boot. La salida seguirá una organización mínima por capas de presentación, lógica de negocio, acceso a datos y persistencia mediante Controller, Service, Repository y Entity. El proyecto generado quedará preparado para trabajar con PostgreSQL. La finalidad no es reemplazar el desarrollo profesional, sino reducir tareas repetitivas y mantener trazabilidad desde el modelo hasta el código base.

1.2 Antecedentes

Las herramientas CASE se desarrollaron para apoyar actividades del ciclo de vida del software mediante automatización, modelado, documentación y generación de artefactos. Productos profesionales como Enterprise Architect muestran el valor de mantener modelos formales en un repositorio, establecer relaciones entre elementos y generar documentación o código a partir de dichos modelos.

El trabajo colaborativo ha incrementado la necesidad de que los artefactos de diseño puedan compartirse y evolucionar sin depender de copias aisladas. En paralelo, la inteligencia artificial generativa y los modelos de lenguaje permiten interpretar instrucciones naturales, explicar conceptos técnicos y producir salidas estructuradas. Esta evolución crea una oportunidad para combinar modelado formal, colaboración y asistencia inteligente dentro de una misma herramienta.

La propuesta toma como referencia las capacidades de modelado profesional, pero concentra el alcance académico en un flujo vertical: proyecto de modelado, diagrama UML, validación, colaboración, transformación a modelo relacional, asistencia mediante IA e ingeniería de generación de backend Spring Boot.

1.3 Planteamiento del problema

En el proceso de diseño de software, los equipos suelen separar el modelado, la documentación, la colaboración y la construcción inicial del backend en herramientas distintas. Esta fragmentación obliga a transferir información manualmente entre diagramas y código, aumenta el riesgo de inconsistencias y dificulta mantener trazabilidad cuando el modelo cambia.

La colaboración sobre diagramas también presenta dificultades. Cuando varios integrantes modifican copias independientes del mismo diseño, deben comparar cambios, resolver diferencias y determinar cuál versión representa el estado vigente. La falta de sincronización y versionado sobre el modelo puede producir pérdida de trabajo o decisiones contradictorias.

Los usuarios con menor experiencia en UML enfrentan una dificultad adicional. Conocer el dominio no garantiza dominar la notación, las relaciones, las

multiplicidades o las reglas de transformación hacia un modelo relacional. Una herramienta que incorpore orientación contextual puede reducir esta barrera sin eliminar el control humano sobre el diseño.

Finalmente, convertir un diagrama de clases en una estructura backend implica repetir tareas mecánicas: crear entidades, repositorios, servicios, controladores y configuraciones. El problema central consiste en la ausencia, dentro del contexto académico planteado, de una herramienta integrada que permita diseñar, colaborar, aprender, transformar y generar una base de software manteniendo un mismo modelo como fuente de información.

1.4 Justificación

1.4.1 Justificación técnica

La propuesta técnica articula el modelado UML, el trabajo colaborativo, la gestión de versiones, la inteligencia artificial, la interoperabilidad y la generación automática de código. Esta articulación permite concebir el modelo como un artefacto estructurado y reutilizable. Toda modificación efectuada desde el lienzo interactivo, mediante comandos procesados por IA o a través de procesos de importación, actualiza de forma consistente un único núcleo de información interna, impidiendo la existencia de fuentes de verdad divergentes.

1.4.2 Justificación académica

El trabajo demanda poner en práctica diversas fases del ciclo de vida del software, abarcando desde la toma de requerimientos y el análisis hasta el diseño, la estructuración arquitectónica, el modelado, la codificación, la verificación y el despliegue. Asimismo, posibilita evidenciar la distinción fundamental entre desarrollar una solución informática bajo estándares formales de calidad y limitarse exclusivamente a escribir código de manera fragmentada.

1.4.3 Justificación de productividad y calidad

La comprobación del diseño, el control de versiones, el trabajo en equipo y la creación automatizada de código disminuyen los reintentos e imprevistos. Por su parte, la

calidad se garantiza mediante la división de responsabilidades, reglas de verificación deterministas, seguimiento continuo de modificaciones y la ejecución de pruebas sobre las transformaciones generadas por la herramienta.

1.4.4 Justificación de innovación

La originalidad del proyecto se centra en fusionar la inteligencia artificial de manera directa con los artefactos propios de la ingeniería de software. La IA no se limitará a mantener un diálogo con el usuario; también estará capacitada para descifrar comandos, sugerir propuestas de reconstrucción a partir de imágenes y fungir como un asistente de aprendizaje, siempre bajo los lineamientos de validación y confirmación establecidos por la plataforma.

1.5 Objetivos

1.5.1 Objetivo general

Desarrollar una plataforma colaborativa de ingeniería de software orientada al diseño conceptual de bases de datos mediante diagramas UML, facilitando el trabajo sincronizado en equipo, la integración de asistencia inteligente y la generación automatizada de una estructura backend con Spring Boot y PostgreSQL a partir del diseño verificado.

1.5.2 Objetivos específicos

1. Recolectar información sobre el proceso de modelado UML, las necesidades de colaboración, las funciones esperadas de una herramienta CASE y los requisitos de generación de backend mediante entrevistas, revisión documental y análisis de herramientas de referencia.
2. Analizar la información recolectada para identificar actores, casos de uso, requisitos funcionales, requisitos no funcionales, riesgos y reglas del producto.
3. Diseñar la arquitectura de la herramienta, el metamodelo interno, el repositorio de proyectos, los mecanismos de colaboración, versionado, inteligencia artificial, interoperabilidad y generación.
4. Implementar un editor UML orientado a diagramas de clases que permita gestionar clases, atributos, operaciones, relaciones y multiplicidades.

5. Implementar mecanismos de trabajo colaborativo que permitan compartir proyectos, sincronizar cambios, mantener historial y resolver conflictos básicos de edición.
6. Integrar inteligencia artificial para interpretar comandos de texto o voz, reconstruir modelos desde imágenes y proporcionar asistencia de aprendizaje y uso contextual.
7. Transformar el modelo UML a una representación relacional y generar una estructura inicial de backend Spring Boot con Controller, Service, Repository y Entity, preparada para PostgreSQL.
8. Validar la calidad del producto mediante pruebas funcionales, de integración, colaboración, transformación, interoperabilidad, generación y aceptación.

1.7 Alcance

La entrega inicial se concibe como un producto mínimo viable que abarca todo el ciclo de trabajo, partiendo desde el establecimiento de un proyecto de modelado hasta la producción completa del backend. El enfoque delimitado otorga prioridad al desarrollo exhaustivo de los diagramas de clases y la edición colaborativa por encima de la cobertura global de diagramas UML.

1.7.1 Alcance funcional

- Autenticación de usuarios y administración integral de proyectos de modelado.
- Elaboración y modificación interactiva de diagramas de clases bajo la especificación UML.
- Comprobación automática de identificadores, datatypes, asociaciones, cardinalidades y pautas arquitectónicas fundamentales.
- Mapeo y conversión del diseño UML hacia un esquema relacional.
- Gestión de acceso compartido a proyectos y coedición simultánea en tiempo casi real.
- Registro histórico de ediciones, control de versiones y reversión a estados previos.
- Gestión esencial de divergencias durante la edición concurrente.
- Procesamiento de peticiones por voz o texto mediante inteligencia artificial para modificar el esquema.

- Interpretación automatizada y digitalización de diagramas UML partiendo de archivos de imagen.
- Asistente virtual de IA para el acompañamiento educativo, clarificación de conceptos UML y apoyo situacional.
- Carga y extracción de los componentes UML admitidos mediante el formato estándar XMI.
- Ajuste de parámetros y construcción estructurada de un backend en Spring Boot mediante capas.
- Descarga del código base generado con soporte preconfigurado para bases de datos PostgreSQL.

1.7.2 Fuera de alcance de la primera versión

- Cobertura completa de todos los diagramas definidos por UML 2.5.1.
- Compatibilidad total con todas las extensiones propietarias de Enterprise Architect.
- Generación automática de reglas de negocio específicas que no estén expresadas en el modelo.
- Reemplazo del desarrollador por la inteligencia artificial.
- Resolución avanzada de conflictos distribuida al nivel de productos comerciales de edición colaborativa.
- Despliegue automático a producción del backend generado.

2. FUNDAMENTACIÓN TEÓRICA

2.1. Ingeniería de Software Asistida por Computadora (CASE)

2.1.1 Concepto

CASE, Computer-Aided Software Engineering, comprende herramientas y técnicas destinadas a apoyar actividades del proceso de software mediante automatización. Su finalidad consiste en mejorar productividad, consistencia, documentación y control de artefactos a lo largo del ciclo de vida. Una herramienta CASE puede apoyar requisitos, análisis, modelado, diseño, generación de código, pruebas, documentación o mantenimiento, según su alcance (Pressman & Maxim, 2020).

2.1.2 Clasificación tradicional

- Upper CASE: orientada a actividades tempranas como requisitos, análisis y diseño.
- Lower CASE: orientada a construcción, pruebas y mantenimiento.
- I-CASE o CASE integrada: combina actividades de varias etapas y mantiene información compartida entre ellas.

2.1.3 Funciones relevantes

- Edición y administración de modelos.
- Repositorio de elementos y relaciones.
- Validación de consistencia.
- Trazabilidad entre artefactos.
- Generación de documentación o código.
- Ingeniería directa y, en algunos productos, ingeniería inversa.
- Intercambio de modelos mediante formatos estándar o propietarios.

2.1.4 Relación con el proyecto

La plataforma propuesta se aproxima a una herramienta CASE integrada de alcance académico porque conecta modelado UML, colaboración, validación, transformación de datos, asistencia inteligente y generación de backend. El valor central reside en

conservar un modelo estructurado que pueda ser consumido por distintas funciones sin duplicar información.

2.2 Desarrollo de Software Basado en Componentes

2.2.1 Concepto de componente

El desarrollo basado en componentes construye sistemas mediante unidades de software con responsabilidades delimitadas, interfaces conocidas y bajo acoplamiento. Un componente encapsula una capacidad y expone contratos para colaborar con otros componentes. El objetivo es favorecer reutilización, sustitución, mantenibilidad y evolución controlada (Sommerville, 2016).

2.2.2 Principios aplicables

- Alta cohesión dentro de cada componente.
- Bajo acoplamiento entre componentes.
- Interfaces explícitas y estables.
- Ocultamiento de detalles internos.
- Reutilización de componentes o plantillas cuando resulte conveniente.
- Composición mediante dependencias controladas.

2.2.3 Aplicación al proyecto

La herramienta se diseñará mediante componentes separados para espacio de trabajo, modelado, validación, colaboración, versionado, IA, intercambio XMI y generación. Esta separación permite modificar el proveedor de IA o la estrategia de persistencia sin alterar el núcleo del metamodelo. El backend generado también seguirá una descomposición por responsabilidades mediante Controller, Service, Repository y Entity.

2.3 Arquitectura de Software

2.3.1 Concepto

La arquitectura de software describe estructuras, elementos, relaciones y decisiones relevantes para construir y evolucionar una solución. ISO/IEC/IEEE 42010:2022

establece conceptos para describir arquitecturas mediante vistas, puntos de vista y modelos. En términos prácticos, una arquitectura permite razonar sobre responsabilidades, dependencias, restricciones y atributos de calidad.

2.3.2 Atributos de calidad relevantes

- Mantenibilidad: facilitar cambios localizados.
- Modificabilidad: permitir sustituir integraciones o reglas sin reconstruir todo el producto.
- Interoperabilidad: intercambiar modelos y datos con formatos definidos.
- Rendimiento: mantener edición fluida y propagación oportuna de cambios.
- Seguridad: proteger proyectos, sesiones y acceso a servicios externos.
- Disponibilidad: mantener acceso al espacio de trabajo durante las sesiones de modelado.
- Testabilidad: permitir validar componentes y transformaciones de manera aislada.

2.3.3 Arquitectura por capas

La arquitectura por capas organiza el software según responsabilidades. En el backend generado se utilizarán cuatro capas de software: Controller para interacción HTTP, Service para coordinación y reglas extensibles, Repository para acceso a persistencia y Entity para representación persistente. PostgreSQL constituye la infraestructura de base de datos con la que interactúa la capa de acceso a datos.

2.3.4 Relación con el proyecto

La herramienta tendrá una arquitectura propia distinta de la arquitectura que genera. La primera necesita editor, colaboración, versionado, IA y motor de transformación. La segunda corresponde al backend Spring Boot producido como salida. Separar ambas evita confundir la base de datos interna de la herramienta con la base de datos del sistema que el usuario está modelando.

2.4 UML 2.5.1, OMG y arquitectura de cuatro niveles

2.4.1 UML

UML es un lenguaje estandarizado para especificar, visualizar y documentar modelos de software. La versión formal publicada por Object Management Group es UML 2.5.1. La especificación define conceptos estructurales y de comportamiento, además de documentos legibles por máquina para su metamodelo. El proyecto se concentrará en el subconjunto necesario para diagramas de clases: clases, propiedades, operaciones, asociaciones, generalizaciones, agregaciones, composiciones y multiplicidades.

2.4.2 Object Management Group

OMG es el consorcio responsable de especificaciones como UML, MOF y XMI. Estas especificaciones permiten tratar un diagrama como una representación de un modelo formal, no como una imagen sin estructura.

2.4.3 Arquitectura de cuatro niveles de metamodelado

Para explicar la relación entre modelos y metamodelos se emplea la arquitectura M0-M3. M3 contiene el meta-metamodelo que permite definir metamodelos, como MOF. M2 contiene el metamodelo UML con conceptos como Class, Property y Association. M1 corresponde al modelo creado por el usuario, por ejemplo las clases Cliente y Venta. M0 corresponde a instancias concretas del dominio representado por ese modelo.

Nivel	Contenido	Ejemplo aplicado
M3	Meta-metamodelo	MOF define mecanismos para construir metamodelos.
M2	Metamodelo	UML define Class, Property, Operation, Association y otros metaelementos.
M1	Modelo	El usuario crea Cliente, Producto, Venta y sus relaciones.
M0	Instancias	Objetos concretos del sistema modelado.

2.4.4 XMI e interoperabilidad

XMI, XML Metadata Interchange, es una especificación de OMG para intercambio de metadatos y modelos basados en MOF. La versión 2.5.1 constituye la referencia para la interoperabilidad prevista. La plataforma implementará un subconjunto controlado de XMI para importar y exportar los elementos UML que soporte, sin afirmar compatibilidad completa con todas las extensiones propietarias de herramientas externas.

2.4.5 Aplicación en la herramienta

El editor deberá conservar un metamodelo interno coherente con los conceptos UML seleccionados. Tanto el lienzo, como la IA, la importación XMI, la colaboración y el generador de backend deben operar sobre esa representación estructurada. Esta decisión resulta esencial para evitar que la lógica dependa de coordenadas o imágenes del diagrama.

2.5 Técnica de Modelado de Objetos (TMO/OMT)

2.5.1 Definición

En esta documentación se emplea TMO como denominación en español de la Técnica de Modelado de Objetos, conocida internacionalmente como OMT, Object Modeling Technique. Rumbaugh y sus colaboradores desarrollaron esta metodología para análisis y diseño orientado a objetos. Su enfoque influyó en la evolución posterior de UML.

2.5.2 Modelos de TMO

- Modelo de objetos: representa la estructura estática mediante clases, atributos, operaciones y relaciones.
- Modelo dinámico: representa estados, eventos, transiciones y comportamiento temporal.
- Modelo funcional: representa transformaciones y flujo de información.

2.5.3 Fases generales

TMO contempla actividades de análisis, diseño del sistema, diseño de objetos e implementación. Para este proyecto interesa especialmente el modelo de objetos, porque el editor se concentra en la estructura de clases y relaciones. Estudiar TMO ayuda a comprender el origen de varios conceptos que posteriormente fueron unificados en UML.

2.5.4 Relación con el proyecto

La plataforma adopta conceptos estructurales compatibles con el modelado orientado a objetos y los representa mediante UML actual. TMO se utiliza como fundamento histórico y metodológico, mientras UML 2.5.1 será la notación formal de referencia para los modelos creados por el usuario.

2.6 Inteligencia Artificial en el Desarrollo de Software

2.6.1 Aplicación en ingeniería de software

La inteligencia artificial se aplica a tareas como asistencia en programación, análisis de texto, generación de pruebas, explicación de código, documentación, búsqueda semántica y apoyo al modelado. En una herramienta de ingeniería, la IA aporta valor cuando trabaja con contexto técnico y produce salidas que el sistema puede verificar antes de aplicarlas.

2.6.2 Modelos de lenguaje y lenguaje natural

Los modelos de lenguaje permiten interpretar instrucciones expresadas por el usuario. Para que una instrucción pueda modificar un diagrama, la respuesta de IA debe transformarse a una operación estructurada. Por ejemplo, la frase “crear una clase Cliente con nombre y correo” debe convertirse en una orden interna con tipo de operación, nombre de clase y lista de atributos, y luego pasar por validaciones deterministas.

2.6.3 Agentes de IA

Un agente de IA combina un modelo de razonamiento o lenguaje con contexto, herramientas y reglas para ayudar a alcanzar un objetivo. En este proyecto el agente estará integrado en la aplicación y tendrá una función educativa y operativa. Podrá explicar cómo realizar una tarea, interpretar el estado del modelo, responder preguntas sobre UML y orientar el siguiente paso del usuario.

2.6.4 Funciones de IA en el proyecto

- Modificar modelos mediante instrucciones de texto o voz.
- Reconstruir una propuesta de modelo a partir de una imagen UML.
- Explicar conceptos UML relacionados con la tarea actual.
- Enseñar el uso de la plataforma mediante instrucciones paso a paso.
- Ofrecer ayuda contextual basada en el proyecto abierto.
- Proponer correcciones, sin reemplazar las validaciones deterministas del motor UML.

2.6.5 Control y calidad de las operaciones inteligentes

Las respuestas de IA no deben modificar directamente la persistencia sin controles. La aplicación debe validar estructura, permisos y consecuencias antes de aplicar una operación. Las acciones destructivas, como eliminar una clase con relaciones, deben requerir confirmación. La trazabilidad debe registrar la instrucción y el cambio finalmente aplicado.

2.7 Spring Boot

2.7.1 Concepto

Spring Boot facilita la construcción de aplicaciones Java independientes basadas en Spring mediante configuración inicial reducida, dependencias de inicio y auto-configuración. La documentación oficial vigente mantiene Spring Boot 4.1.1 como una versión estable al momento de esta revisión. El proyecto utilizará Spring Boot como tecnología objetivo del generador, no necesariamente como tecnología interna de la propia herramienta.

2.7.2 Elementos relevantes para el generador

- Spring Web para controladores REST.
- Inyección de dependencias para conectar servicios y componentes.
- Spring Data JPA para repositorios.
- Jakarta Persistence para entidades y relaciones.
- Configuración externalizada mediante application.properties o application.yml.
- Maven o Gradle para dependencias y construcción.

2.7.3 Estructura backend generada

Capa	Responsabilidad	Artefacto esperado
Controller	Recibir solicitudes HTTP y exponer recursos REST.	Clases Controller.
Service	Coordinar reglas y operaciones de aplicación.	Clases o interfaces Service.
Repository	Abstraer operaciones de persistencia.	Interfaces Repository basadas en Spring Data JPA.
Entity	Representar información persistente y relaciones.	Clases Entity con anotaciones Jakarta Persistence.
Infraestructura de datos	Almacenar la información del backend generado.	Configuración de conexión a PostgreSQL.

2.7.4 Relación con el proyecto

El generador deberá traducir clases, atributos, identificadores, tipos y multiplicidades hacia estructuras Java y mapeos de persistencia. El código generado se considera una base extensible. El equipo desarrollador seguirá siendo responsable de completar reglas de negocio, seguridad, validaciones específicas, pruebas y decisiones de despliegue.

3. PROCESO DE DESARROLLO

3.1 Flujo de trabajo: Captura de requisitos

La captura de requisitos define las funciones que el producto debe ofrecer y las restricciones que condicionan su construcción. Para mantener el enfoque de herramienta colaborativa, se priorizan los objetivos de ingeniería de software: modelar, compartir, validar, transformar, versionar, recibir asistencia y generar artefactos.

3.1.1 Identificar actores y casos de uso

3.1.1.1 Actores Principales

ID	Actor	Tipo	Responsabilidad
A1	Usuario desarrollador	Actor principal	Especialista que emplea el entorno compartido para llevar a cabo tareas de ingeniería de software.
A2	Servicio externo de inteligencia artificial	Actor externo	Sistema informático encargado de brindar soluciones e intervenciones inteligentes al sistema.
A3	Herramienta externa UML	Actor externo	Software de diseño adaptado a estándares UML que facilita la transferencia de esquemas mediante recursos como XMI.

3.1.1.2 Casos de Uso

ID	Caso de uso
CU01	Iniciar sesión y gestionar cuenta
CU02	Organizar espacios de trabajo y diagramas
CU03	Invitar miembros y asignar roles
CU04	Construir diagramas UML
CU05	Auditar errores en el diagrama
CU06	Convertir UML a base de datos relacional
CU07	Coeditar diagramas en vivo
CU08	Fusionar cambios concurrentes
CU09	Restaurar versiones y auditar cambios
CU10	Refactorizar diagrama con instrucciones de IA
CU11	Digitalizar diagrama desde captura o boceto
CU12	Solicitar sugerencias al copiloto de diseño

CU13	Cargar y descargar esquemas (XMI / JSON / XML)
CU14	Definir parámetros de arquitectura y código
CU15	Construir código fuente Spring Boot

3.1.2 Priorización de casos de uso

ID	Prioridad	Riesgo	Justificación	Ciclo
CU01	Alta	Normal	Soporta identidad y acceso a proyectos.	1
CU02	Crítica	Normal	Define la unidad central de trabajo.	1
CU03	Alta	Normal	Habilita colaboración sobre proyectos.	2
CU04	Crítica	Alta	Es el núcleo funcional del editor.	1
CU05	Crítica	Alta	Protege transformaciones y generación.	1
CU06	Alta	Alta	Conecta UML con diseño relacional.	1
CU07	Crítica	Alta	Demuestra el carácter colaborativo.	2
CU08	Alta	Alta	Evita pérdida de trabajo concurrente.	2
CU09	Alta	Normal	Aporta trazabilidad y recuperación.	2
CU10	Crítica	Alta	Demuestra IA operando sobre el modelo.	3
CU11	Alta	Alta	Permite reconstrucción asistida desde imágenes.	3
CU12	Alta	Normal	Materializa el agente de aprendizaje.	2
CU13	Alta	Alta	Aporta interoperabilidad mediante XMI.	3
CU14	Alta	Normal	Controla parámetros de la salida generada.	3
CU15	Crítica	Alta	Cierra el flujo modelo a software.	3

3.1.3 Detallar casos de uso

CU01. Iniciar sesión y gestionar cuenta

Campo	**Descripción**
Caso de uso	CU01 - Iniciar sesión y gestionar cuenta

Propósito	Permitir que el usuario desarrollador pueda registrarse, autenticarse y acceder a la plataforma para utilizar las funcionalidades de modelado y colaboración.
Resumen	Permite crear una cuenta, iniciar sesión y mantener una sesión válida dentro de la herramienta colaborativa.
Actor principal	Usuario desarrollador
Actor iniciador	Usuario desarrollador
Precondiciones	El usuario debe contar con conexión a la plataforma y proporcionar información válida para autenticarse.
Flujo principal	1. El usuario ingresa a la plataforma. 2. Selecciona la opción de registro o inicio de sesión. 3. Introduce sus datos de acceso. 4. La plataforma valida la información proporcionada. 5. El sistema permite el acceso al espacio de trabajo.
Flujos alternativos / excepciones	1. Si los datos ingresados son incorrectos, la plataforma informa el error y solicita una nueva autenticación. 2. Si el usuario no posee una cuenta, debe completar el proceso de registro.
Postcondiciones	El usuario obtiene acceso a la plataforma y puede utilizar las funcionalidades disponibles según sus permisos.

CU02. Organizar espacios de trabajo y diagramas

Campo	**Descripción**
Caso de uso	CU02 - Organizar espacios de trabajo y diagramas
Propósito	Administrar los espacios de trabajo donde se almacenan modelos UML, configuraciones y elementos relacionados con el desarrollo del software.
Resumen	Permite crear, abrir, modificar, guardar, duplicar y eliminar proyectos de modelado.
Actor principal	Usuario desarrollador
Actor iniciador	Usuario desarrollador autenticado
Precondiciones	El usuario debe tener una sesión válida dentro de la plataforma.
Flujo principal	1. El usuario ingresa al módulo de proyectos. 2. Crea un nuevo proyecto o selecciona uno existente. 3. La plataforma carga la información asociada al proyecto. 4. El usuario realiza modificaciones permitidas sobre el proyecto. 5. La plataforma guarda los cambios realizados.

Flujos alternativos / excepciones	1. Si el proyecto solicitado no existe, la plataforma informa la situación. 2. Si el usuario no tiene permisos suficientes, se restringe la operación solicitada.
Postcondiciones	El proyecto queda creado, actualizado, abierto, duplicado o eliminado según la acción realizada.

CU03. Invitar miembros y asignar roles

Campo	**Descripción**
Caso de uso	CU03 - Invitar miembros y asignar roles
Propósito	Permitir que los usuarios desarrolladores trabajen sobre proyectos compartidos mediante mecanismos de colaboración.
Resumen	Permite compartir proyectos con otros usuarios y gestionar participantes del espacio colaborativo.
Actor principal	Usuario desarrollador
Actor iniciador	Usuario desarrollador autenticado
Precondiciones	El usuario debe tener un proyecto creado dentro de la plataforma.
Flujo principal	1. El usuario selecciona un proyecto. 2. Solicita compartir el proyecto con otro usuario. 3. La plataforma registra la participación del nuevo integrante. 4. Los usuarios autorizados pueden acceder al proyecto compartido.
Flujos alternativos / excepciones	1. Si el usuario invitado no existe, la plataforma solicita registrar un usuario válido. 2. Si el usuario no posee autorización sobre el proyecto, se rechaza la acción.
Postcondiciones	El proyecto queda disponible para los usuarios autorizados.

CU04. Construir diagramas UML

Campo	**Descripción**
Caso de uso	CU04 - Construir diagramas UML
Propósito	Permitir al usuario desarrollador crear y modificar modelos UML orientados al diseño conceptual de software y bases de datos.

Resumen	Permite crear elementos UML como clases, atributos, operaciones y relaciones entre componentes del modelo.
Actor principal	Usuario desarrollador
Actor iniciador	Usuario desarrollador autenticado
Precondiciones	El usuario debe tener un proyecto abierto dentro de la plataforma.
Flujo principal	1. El usuario ingresa al editor UML. 2. Crea elementos del modelo como clases y atributos. 3. Define relaciones entre las clases creadas. 4. Configura propiedades del modelo. 5. La plataforma almacena los cambios realizados.
Flujos alternativos / excepciones	1. Si el usuario intenta crear elementos incompletos, la plataforma solicita completar la información requerida. 2. Si existe un conflicto en la estructura del modelo, el sistema informa la inconsistencia detectada.
Postcondiciones	El modelo UML queda creado o actualizado dentro del proyecto seleccionado.

CU05. Auditar errores en el diagrama

Campo	**Descripción**
Caso de uso	CU05 - Auditar errores en el diagrama
Propósito	Verificar que el modelo UML diseñado cumpla condiciones básicas de consistencia antes de continuar con otras etapas del desarrollo.
Resumen	Permite analizar el modelo para identificar errores estructurales o elementos incompletos.
Actor principal	Usuario desarrollador
Actor iniciador	Usuario desarrollador
Precondiciones	Debe existir un modelo UML creado dentro del proyecto.
Flujo principal	1. El usuario solicita la validación del modelo. 2. La plataforma analiza clases, atributos y relaciones. 3. El sistema identifica posibles inconsistencias. 4. La plataforma muestra los resultados de validación. 5. El usuario realiza las correcciones necesarias.
Flujos alternativos / excepciones	1. Si el modelo no contiene errores, la plataforma muestra un resultado satisfactorio. 2. Si existen elementos incompletos, el sistema indica los componentes que requieren modificación.
Postcondiciones	El usuario obtiene información sobre el estado de calidad del modelo UML.

CU06. Convertir UML a base de datos relacional

Campo	Descripción
Caso de uso	CU06 - Convertir UML a base de datos relacional
Propósito	Transformar el modelo conceptual UML en una estructura orientada al diseño de bases de datos relacionales.
Resumen	Permite convertir clases, atributos y relaciones UML en entidades, tablas y relaciones del modelo relacional.
Actor principal	Usuario desarrollador
Actor iniciador	Usuario desarrollador
Precondiciones	Debe existir un modelo UML validado o con una estructura compatible.
Flujo principal	1. El usuario solicita generar el modelo relacional. 2. La plataforma analiza las clases y relaciones existentes. 3. El sistema transforma los elementos UML en estructuras relacionales. 4. Se generan tablas, claves y relaciones correspondientes. 5. La plataforma muestra el resultado generado.
Flujos alternativos / excepciones	1. Si existen relaciones ambiguas, la plataforma solicita una revisión del modelo. 2. Si faltan elementos necesarios, se informa al usuario antes de realizar la transformación.
Postcondiciones	El modelo relacional queda generado a partir del diseño UML.

CU07. Coeditar diagramas en vivo

Campo	Descripción
Caso de uso	CU07 - Coeditar diagramas en vivo
Propósito	Permitir que varios usuarios desarrolladores trabajen sobre un mismo modelo de forma simultánea.
Resumen	Permite visualizar y aplicar modificaciones realizadas por diferentes participantes dentro de un proyecto compartido.
Actor principal	Usuario desarrollador
Actor iniciador	Usuario desarrollador
Precondiciones	Debe existir un proyecto compartido con usuarios autorizados.
Flujo principal	1. El usuario ingresa a un proyecto colaborativo.2. Realiza modificaciones sobre el modelo UML.3. La plataforma registra los cambios realizados.4. Los demás usuarios

	visualizan las actualizaciones.5. El modelo compartido queda sincronizado.
Flujos alternativos / excepciones	1. Si dos usuarios modifican el mismo elemento simultáneamente, la plataforma detecta la situación.2. Si existe pérdida de conexión, los cambios pendientes se almacenan temporalmente.
Postcondiciones	El modelo queda actualizado con las modificaciones realizadas por los participantes.

CU08. Fusionar cambios concurrentes

Campo	**Descripción**
Caso de uso	CU08 - Fusionar cambios concurrentes
Propósito	Resolver situaciones donde diferentes usuarios realizan modificaciones incompatibles sobre un mismo modelo.
Resumen	Permite identificar conflictos y seleccionar la versión correcta de los cambios realizados.
Actor principal	Usuario desarrollador
Actor iniciador	Usuario desarrollador
Precondiciones	Debe existir una edición colaborativa activa.
Flujo principal	1. La plataforma detecta modificaciones simultáneas. 2. El sistema identifica los elementos afectados. 3. Muestra las versiones en conflicto. 4. El usuario selecciona la modificación correspondiente. 5. La plataforma actualiza el modelo final.
Flujos alternativos / excepciones	1. Si no existe conflicto, los cambios se aplican automáticamente. 2. Si el usuario cancela la resolución, se mantiene el estado anterior.
Postcondiciones	El modelo queda actualizado sin inconsistencias por modificaciones simultáneas.

CU09. Restaurar versiones y auditar cambios

Campo	**Descripción**
Caso de uso	CU09 - Restaurar versiones y auditar cambios
Propósito	Permitir controlar la evolución histórica de los modelos desarrollados dentro de la plataforma.
Resumen	Permite registrar cambios, consultar versiones anteriores y recuperar estados previos del modelo.

Actor principal	Usuario desarrollador
Actor iniciador	Usuario desarrollador
Precondiciones	Debe existir un modelo almacenado en la plataforma.
Flujo principal	1. El usuario consulta el historial del modelo. 2. La plataforma muestra las versiones disponibles. 3. El usuario selecciona una versión. 4. El sistema permite visualizar o restaurar el estado seleccionado.
Flujos alternativos / excepciones	1. Si no existen versiones anteriores, la plataforma informa que únicamente existe la versión actual. 2. Si una versión está protegida, se restringe su modificación.
Postcondiciones	El usuario obtiene control sobre la evolución del modelo.

CU10. Refactorizar diagrama con instrucciones de IA

Campo	Descripción
Caso de uso	CU10 - Refactorizar diagrama con instrucciones de IA
Propósito	Permitir modificar modelos UML mediante instrucciones escritas en lenguaje natural.
Resumen	El usuario proporciona una instrucción y la inteligencia artificial interpreta la acción requerida para modificar el modelo.
Actor principal	Usuario desarrollador
Actor secundario	Servicio externo de inteligencia artificial
Actor iniciador	Usuario desarrollador
Precondiciones	Debe existir un modelo UML abierto y disponibilidad del servicio de IA.
Flujo principal	1. El usuario introduce una instrucción mediante texto. 2. La plataforma envía la solicitud al servicio de IA. 3. La IA interpreta la intención del usuario. 4. La plataforma transforma la respuesta en acciones sobre el modelo. 5. El modelo UML se actualiza.
Flujos alternativos / excepciones	1. Si la instrucción no es clara, la plataforma solicita una aclaración. 2. Si el servicio de IA no está disponible, se informa al usuario.
Postcondiciones	El modelo queda modificado según la instrucción interpretada.

CU11. Digitalizar diagrama desde captura o boceto

Campo	Descripción
Caso de uso	CU11 - Digitalizar diagrama desde captura o boceto
Propósito	Permitir transformar imágenes de diagramas UML en modelos digitales editables dentro de la plataforma.
Resumen	El usuario carga una imagen de un diagrama UML y la inteligencia artificial analiza sus elementos para reconstruir clases, atributos y relaciones dentro del editor.
Actor principal	Usuario desarrollador
Actor secundario	Servicio externo de inteligencia artificial
Actor iniciador	Usuario desarrollador
Precondiciones	El usuario debe tener un proyecto abierto y disponer de una imagen válida de un diagrama UML.
Flujo principal	1. El usuario selecciona la opción de importar diagrama mediante imagen. 2. Carga la imagen del modelo UML. 3. La plataforma envía la imagen al servicio de inteligencia artificial. 4. El servicio analiza los elementos gráficos del diagrama. 5. La plataforma recibe la información procesada. 6. El sistema genera los elementos UML identificados dentro del editor. 7. El usuario revisa y ajusta el modelo generado.
Flujos alternativos / excepciones	1. Si la imagen tiene baja calidad, la plataforma solicita una imagen con mejor resolución. 2. Si la inteligencia artificial no identifica correctamente un elemento, el usuario puede modificarlo manualmente. 3. Si el formato de imagen no es compatible, el sistema rechaza la carga.
Postcondiciones	El modelo UML queda reconstruido digitalmente y disponible para edición dentro del proyecto.

CU12. Solicitar sugerencias al copiloto de diseño

Campo	Descripción
Caso de uso	CU12 - Solicitar sugerencias al copiloto de diseño
Propósito	Proporcionar orientación al usuario durante el aprendizaje y utilización correcta de la plataforma.

Resumen	Permite que el usuario consulte dudas, reciba explicaciones y obtenga recomendaciones relacionadas con la herramienta, UML y el proceso de desarrollo de software.
Actor principal	Usuario desarrollador
Actor secundario	Servicio externo de inteligencia artificial
Actor iniciador	Usuario desarrollador
Precondiciones	El usuario debe encontrarse dentro de la plataforma y contar con acceso al asistente inteligente.
Flujo principal	1. El usuario realiza una consulta al agente inteligente. 2. La plataforma analiza el contexto actual del proyecto y la actividad realizada. 3. La solicitud es procesada mediante el servicio de inteligencia artificial. 4. El agente genera una respuesta orientada al usuario. 5. El usuario recibe instrucciones, explicaciones o recomendaciones.
Flujos alternativos / excepciones	1. Si la consulta no es suficientemente clara, el agente solicita información adicional. 2. Si el servicio de IA no está disponible, la plataforma informa la limitación temporal. 3. Si la consulta está fuera del alcance del sistema, el agente indica la situación.
Postcondiciones	El usuario obtiene asistencia para comprender y utilizar correctamente las funcionalidades de la plataforma.

CU13. Cargar y descargar esquemas (XMI / JSON / XML)

Campo	**Descripción**
Caso de uso	CU13 -Cargar y descargar esquemas (XMI / JSON / XML)
Propósito	Permitir el intercambio de modelos UML con otras herramientas mediante formatos compatibles y estándares de modelado.
Resumen	Permite cargar modelos existentes o exportar diseños creados dentro de la plataforma para facilitar la interoperabilidad con otras herramientas CASE.
Actor principal	Usuario desarrollador
Actor secundario	Herramienta externa UML
Actor iniciador	Usuario desarrollador
Precondiciones	El usuario debe tener un proyecto abierto y contar con un archivo compatible o un modelo creado dentro de la plataforma.
Flujo principal	1. El usuario selecciona la opción importar o exportar modelo.

	2. Para importar, selecciona un archivo compatible. 3. La plataforma interpreta la estructura del modelo recibido. 4. El sistema genera los elementos UML correspondientes. 5. Para exportar, la plataforma transforma el modelo actual a un formato compatible. 6. El usuario obtiene el archivo generado.
Flujos alternativos / excepciones	1. Si el archivo no cumple el formato esperado, la plataforma informa el error. 2. Si existen elementos incompatibles entre herramientas, el sistema muestra advertencias antes de completar la operación. 3. Si la exportación falla, el usuario puede repetir el proceso.
Postcondiciones	El modelo queda importado correctamente o exportado en un formato compatible.

CU14. Definir parámetros de arquitectura y código

Campo	**Descripción**
Caso de uso	CU14 - Definir parámetros de arquitectura y código
Propósito	Permitir al usuario definir los parámetros necesarios antes de generar la estructura del software backend.
Resumen	Permite establecer configuraciones relacionadas con la generación automática del proyecto Spring Boot a partir del modelo diseñado.
Actor principal	Usuario desarrollador
Actor iniciador	Usuario desarrollador
Precondiciones	Debe existir un modelo UML validado y listo para ser transformado en software.
Flujo principal	1. El usuario selecciona la opción de generación de backend. 2. La plataforma muestra las opciones de configuración disponibles. 3. El usuario define parámetros del proyecto. 4. Configura información como nombre del proyecto, paquetes, base de datos y estructura deseada. 5. La plataforma almacena la configuración seleccionada.
Flujos alternativos / excepciones	1. Si existen datos incompletos, la plataforma solicita completar la información requerida. 2. Si la configuración seleccionada no es compatible con el modelo, el sistema muestra una advertencia.
Postcondiciones	La plataforma cuenta con la configuración necesaria para generar el backend.

CU15. Construir código fuente Spring Boot

Campo	Descripción
Caso de uso	CU15 - Construir código fuente Spring Boot
Propósito	Transformar automáticamente un modelo UML diseñado en una estructura inicial de backend basada en Spring Boot.
Resumen	Permite generar componentes de software a partir del modelo UML, incluyendo capas de arquitectura y configuración de persistencia.
Actor principal	Usuario desarrollador
Actor iniciador	Usuario desarrollador
Precondiciones	Debe existir un modelo UML configurado y una parametrización válida para la generación.
Flujo principal	1. El usuario solicita generar el backend. 2. La plataforma analiza las clases y relaciones del modelo UML. 3. El sistema genera la estructura del proyecto Spring Boot. 4. Se crean componentes como Entity, Repository, Service y Controller. 5. Se generan configuraciones necesarias para la conexión con PostgreSQL. 6. La plataforma presenta el proyecto generado al usuario.
Flujos alternativos / excepciones	1. Si el modelo contiene elementos no compatibles, la plataforma informa los componentes que requieren modificación. 2. Si la generación falla, el sistema muestra el error ocurrido. 3. Si el usuario modifica el modelo, debe ejecutar nuevamente la generación para actualizar el resultado.
Postcondiciones	Se obtiene una estructura inicial de backend Spring Boot generada a partir del modelo UML.

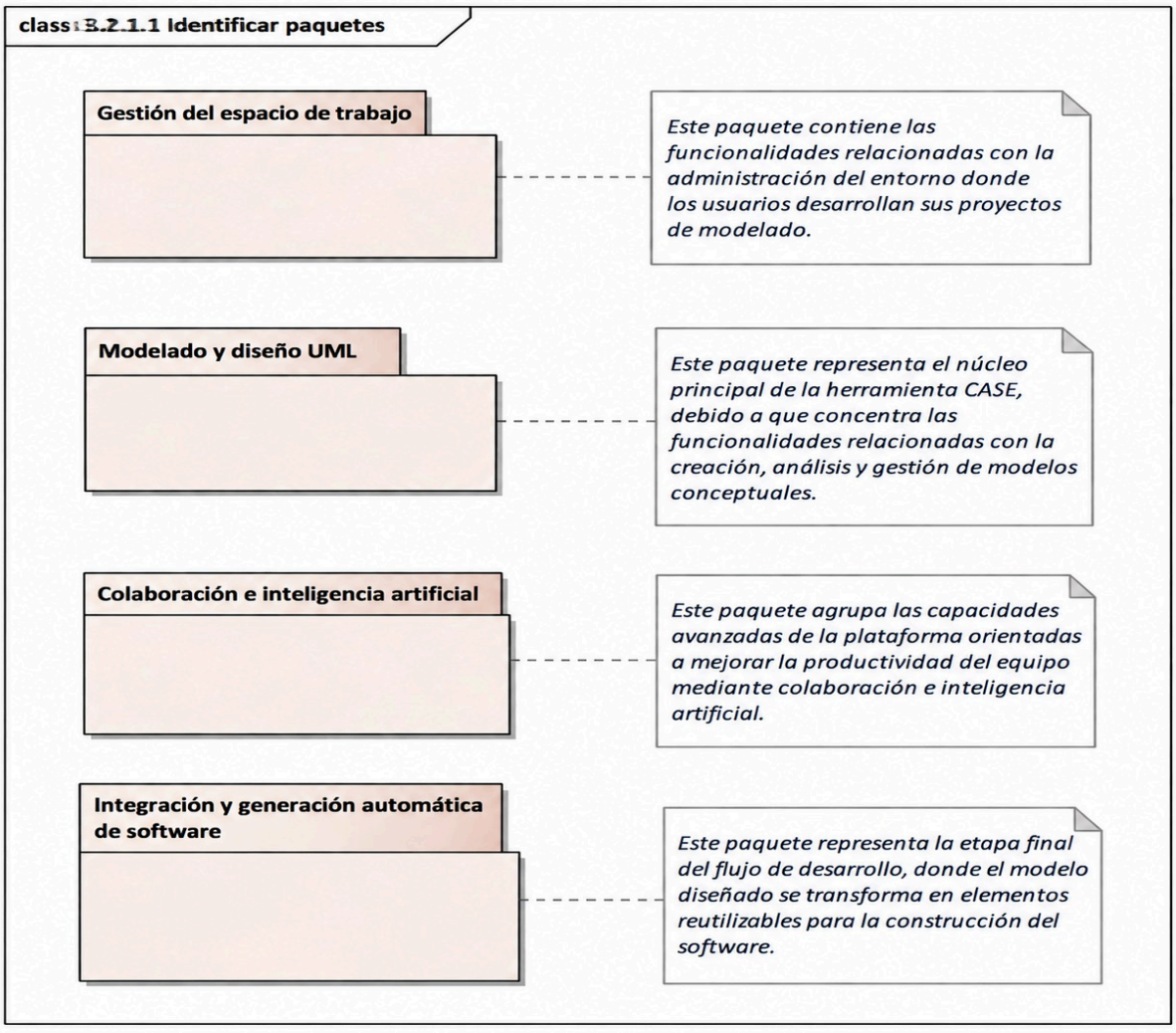
3.1.4 Estructurar casos de uso

3.1.4.1 Diagrama General De Casos De Uso

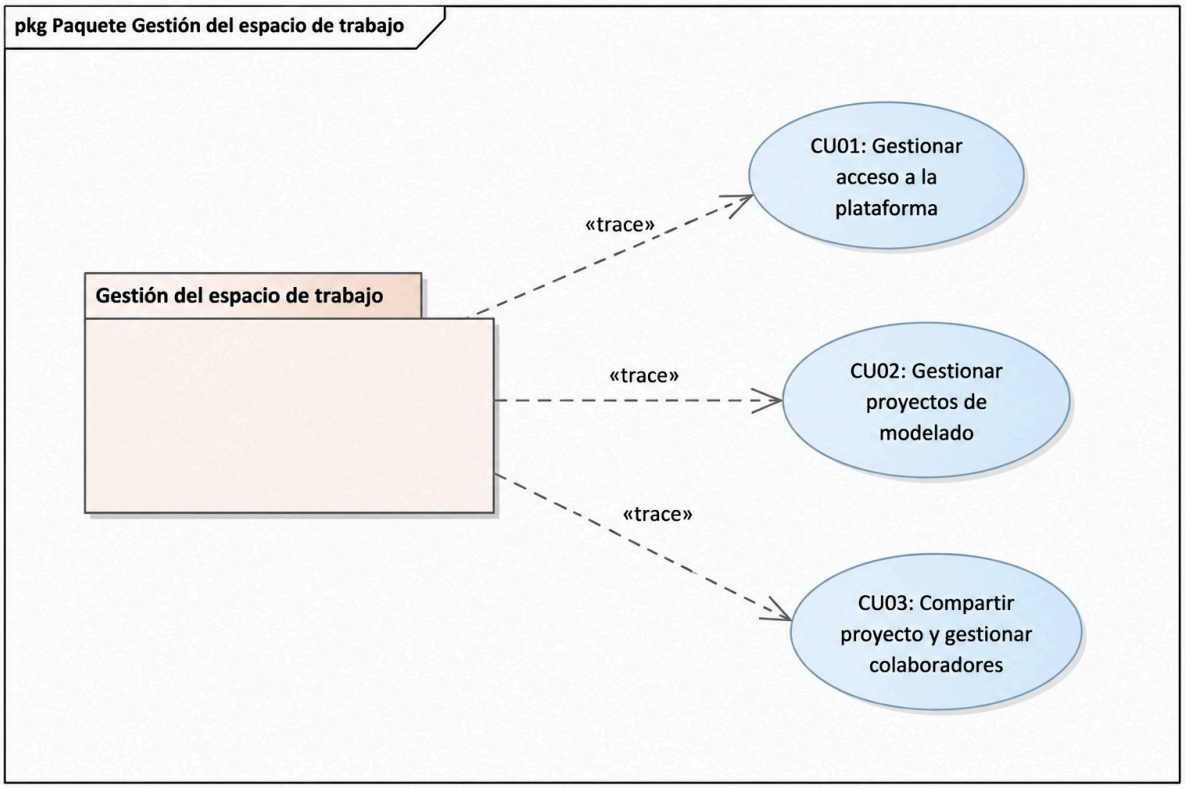
3.2.1 Análisis de paquetes

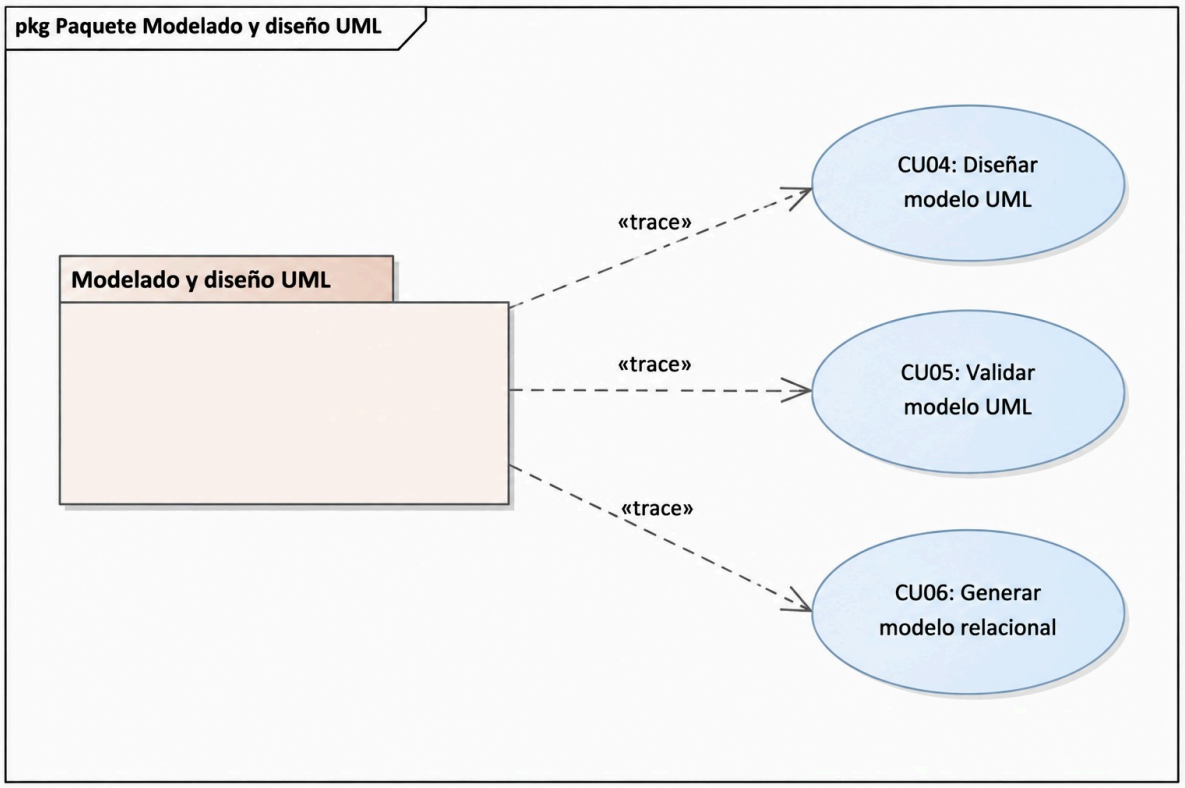
3.2.1.1 Identificar paquetes

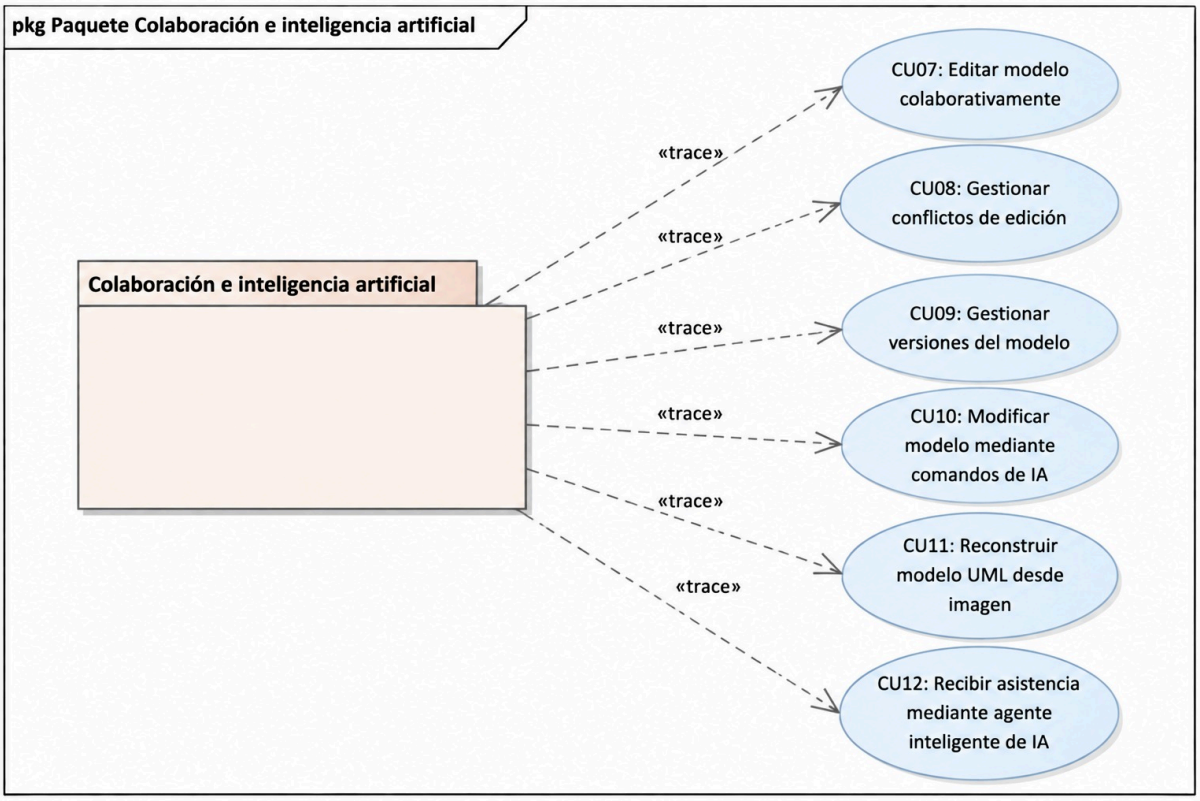
Paquete	Nombre	Casos de uso asignados
Paquete 1	Gestión del espacio de trabajo	CU01, CU02, CU03
Paquete 2	Modelado y diseño UML	CU04, CU05, CU06
Paquete 3	Colaboración e inteligencia artificial	CU07, CU08, CU09, CU10, CU11, CU12
Paquete 4	Integración y generación automática de software	CU13, CU14, CU15

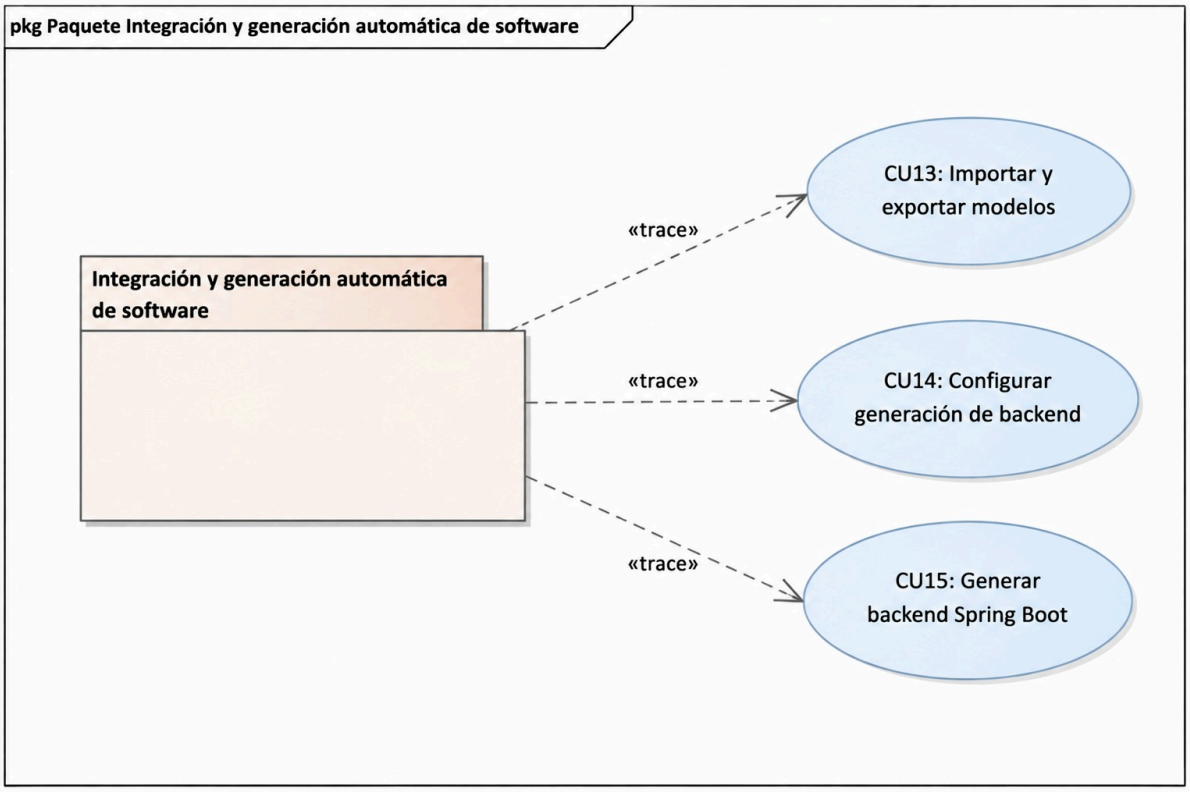


3.2.1.2 Relacionar paquetes con casos de uso






pkg Paquete Colaboración e inteligencia artificial
Colaboración e inteligencia artificial
«trace»
«trace»
«trace»
«trace»
«trace»
«trace»
CU07: Editar modelo colaborativamente
CU08: Gestionar conflictos de edición
CU09: Gestionar versiones del modelo
CU10: Modificar modelo mediante comandos de IA
CU11: Reconstruir modelo UML desde imagen
CU12: Recibir asistencia mediante agente inteligente de IA


pkg Paquete Integración y generación automática de software
Integración y generación automática de software
«trace»
«trace»
«trace»
CU13: Importar y exportar modelos
CU14: Configurar generación de backend
CU15: Generar backend Spring Boot

3.2.1.3 Vista de paquetes

3.2.2 Análisis de casos de uso

Para cada caso de uso se elaborará un diagrama de análisis utilizando clases de frontera, control y entidad cuando el caso lo requiera. Frontera representa la interacción con el usuario o sistema externo. Control coordina el caso de uso. Entidad representa información persistente o conceptos del dominio de la herramienta. Estos elementos pertenecen al análisis y no deben confundirse con Controller, Service, Repository y Entity del backend que la herramienta generará.

DIAGRAMA DE ANÁLISIS CU01 - GESTIONAR ACCESO A LA PLATAFORMA

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU02 - GESTIONAR PROYECTOS DE MODELADO

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU03 - COMPARTIR PROYECTO Y GESTIONAR COLABORADORES

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU04 - DISEÑAR MODELO UML

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU05 - VALIDAR MODELO UML

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU06 - GENERAR MODELO RELACIONAL

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU07 - EDITAR MODELO COLABORATIVAMENTE

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU08 - RESOLVER CONFLICTOS DE EDICIÓN COLABORATIVA

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU09 - GESTIONAR VERSIONES E HISTORIAL DEL MODELO

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU10 - MODIFICAR MODELO MEDIANTE COMANDOS DE IA

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU11 - RECONSTRUIR MODELO UML DESDE IMAGEN

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU12 - RECIBIR ASISTENCIA MEDIANTE AGENTE DE IA

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU13 - IMPORTAR Y EXPORTAR MODELOS

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU14 - CONFIGURAR GENERACIÓN DE BACKEND

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

DIAGRAMA DE ANÁLISIS CU15 - GENERAR BACKEND SPRING BOOT

[Insertar diagrama Boundary-Control-Entity o diagrama de robustez.]

3.2.3 Analizar clases

Ciclo	Clases de análisis preliminares
Ciclo 1	Usuario, Proyecto, ModeloUML, ClaseUML, AtributoUML, OperacionUML, RelacionUML, Multiplicidad, ValidadorModelo, ModeloRelacional, TablaRelacional, ColumnaRelacional.
Ciclo 2	ProyectoColaborador, PermisoProyecto, SesionColaboracion, OperacionModelo, ConflictoEdicion, VersionModelo, EventoHistorial, ConversacionAgente.

Ciclo	Clases de análisis preliminares
Ciclo 3	ComandoIA, ResultadoIA, AnalisisImagen, ImportacionXMI, ExportacionXMI, ConfiguracionGeneracion, ModeloGeneracion, ArtefactoGenerado.

FIGURA 12. DIAGRAMA DE CLASES DE ANÁLISIS - CICLO 1

[Insertar la imagen o diagrama correspondiente en este espacio.]

FIGURA 13. DIAGRAMA DE CLASES DE ANÁLISIS - CICLO 2

[Insertar la imagen o diagrama correspondiente en este espacio.]

FIGURA 14. DIAGRAMA DE CLASES DE ANÁLISIS - CICLO 3

[Insertar la imagen o diagrama correspondiente en este espacio.]

3.2.4 Análisis de paquete

El análisis de paquetes consolidará las clases de frontera, control y entidad, y verificará que cada responsabilidad tenga un único lugar principal. Las dependencias deben orientarse hacia contratos, evitando que colaboración, IA o generación conozcan detalles visuales del lienzo. El metamodelo debe actuar como núcleo compartido.

FIGURA 15. DIAGRAMA DE ANÁLISIS DE PAQUETES Y DEPENDENCIAS

[Insertar la imagen o diagrama correspondiente en este espacio.]

3.3 Flujo de trabajo: Diseño

3.3.1 Diseño de arquitectura

Componente	Responsabilidad
Cliente de modelado	Presentar lienzo UML, paneles, colaboración, historial y agente de asistencia.
API de aplicación	Gestionar acceso, proyectos, permisos y operaciones de aplicación.
Núcleo de modelado	Mantener metamodelo y operaciones estructuradas sobre clases y relaciones.
Motor de validación	Ejecutar reglas deterministas antes de transformación o generación.
Servicio de colaboración	Coordinar sesiones, operaciones y propagación de cambios.
Servicio de versionado	Crear snapshots, historial y restauraciones.
Adaptador de IA	Aislar proveedor externo y convertir respuestas en estructuras validadas.
Motor de transformación	Derivar modelo relacional y modelo intermedio de generación.
Intercambio XMI	Importar y exportar el subconjunto UML soportado.
Generador Spring Boot	Aplicar reglas y plantillas para producir artefactos Java y configuración.
Repositorio interno	Persistir proyectos, modelos, participantes, versiones y configuraciones.

FIGURA 16. DIAGRAMA DE ARQUITECTURA GENERAL DE LA HERRAMIENTA

[Insertar la imagen o diagrama correspondiente en este espacio.]

FIGURA 17. DIAGRAMA UML DE COMPONENTES

[Insertar la imagen o diagrama correspondiente en este espacio.]

3.3.1.1 Diseño de arquitectura física

FIGURA 18. DIAGRAMA UML DE DESPLIEGUE

[Insertar la imagen o diagrama correspondiente en este espacio.]

3.3.1.2 Diseño de arquitectura Logica

3.3.2 Diseño de Datos

3.3.2.1 Diseño de datos logicos

Diagrama De Clases

Mapeo

3.3.2.2 Diseño de Datos Físicos

Script

3.3.3 Diseño de Caso De Uso

3.3.3.1 Diagrama de secuencia

3.4 Flujo de trabajo: Implementación

La construcción se realizará por incrementos. La primera prioridad técnica es el metamodelo y sus operaciones, porque todas las funciones dependen de él. El lienzo visual debe actuar como una vista editable del modelo estructurado. Después se incorporarán validación, colaboración, versionado, IA, interoperabilidad y generación.

3.4.1 Herramienta de Desarrollo Del Software

3.4.2 Implementacion de la arquitectura de sistema

3.4.3 Implementacion de la Arquitectura del Subsistema

3.5 Flujo de trabajo: Pruebas

4. RESULTADOS ESPERADOS

Al finalizar el proyecto se espera disponer de una versión funcional que permita crear un proyecto, diseñar un diagrama de clases, validarlo, colaborar con otros usuarios, mantener historial, utilizar IA para asistencia y modificación, reconstruir un modelo

desde una imagen dentro de las limitaciones definidas, importar o exportar XMI y generar una estructura backend Spring Boot preparada para PostgreSQL.

El resultado académico deberá demostrar trazabilidad entre requisitos, casos de uso, análisis, diseño, implementación y pruebas. La evaluación del producto debe considerar productividad, calidad e innovación, además del funcionamiento visible de la interfaz.

5. CONCLUSIONES

La revisión del proyecto confirma que su núcleo debe modelarse como una herramienta colaborativa de ingeniería de software. Los actores principales no representan cargos administrativos. Representan roles de trabajo sobre proyectos de modelado: usuario general, propietario, colaborador y un servicio externo de IA cuando exista dicha dependencia.

La reducción a quince casos de uso mantiene un alcance controlado sin eliminar las capacidades que definen el producto. El modelado manual, la colaboración, el versionado, la IA, la interoperabilidad y la generación de backend quedan representados como objetivos completos del usuario y no como operaciones fragmentadas.

PUDS proporciona la estructura de ingeniería mediante captura de requisitos, análisis, diseño, implementación y pruebas. Scrum complementa esta estructura organizando el trabajo de las cuatro semanas en incrementos revisables. Ambos enfoques cumplen funciones distintas y pueden coexistir sin mezclar sus conceptos.

El metamodelo interno constituye una decisión arquitectónica central. Si todas las funciones operan sobre una misma representación estructurada, la herramienta podrá mantener consistencia entre el lienzo, la colaboración, la IA, XMI, el modelo relacional y el backend generado.

6. RECOMENDACIONES

- Mantener los quince casos de uso como línea base del producto y evitar crear casos menores para operaciones internas.

- Conservar la generalización entre Usuario de la herramienta, Propietario y Colaborador en el modelo de actores.
- No modelar agentes, motores, bases de datos o capas internas como actores si están dentro del límite del sistema.
- Definir el metamodelo interno antes de profundizar en la interfaz visual.
- Documentar la tecnología real de implementación cuando el equipo la seleccione, sin confundirla con Spring Boot, que corresponde al backend generado.
- Definir desde el inicio el subconjunto de UML y XMI soportado.
- Validar cada transformación con reglas deterministas y usar IA como asistencia controlada.
- Registrar evidencias por ciclo para completar posteriormente todos los espacios marcados en rojo.
- Mantener sincronizados casos de uso, clases, secuencias, paquetes, pruebas y arquitectura cada vez que cambie el alcance.

7. BIBLIOGRAFÍA

Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley.

ISO/IEC/IEEE. (2022). ISO/IEC/IEEE 42010:2022. Software, systems and enterprise - Architecture description.

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley.

Larman, C. (2004). Applying UML and Patterns (3rd ed.). Prentice Hall.

Object Management Group. (2015). XML Metadata Interchange (XMI), Version 2.5.1.

Object Management Group. (2016). Meta Object Facility (MOF), Version 2.5.1.

Object Management Group. (2017). Unified Modeling Language (UML), Version 2.5.1.

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach. McGraw-Hill.

Rumbaugh, J., Blaha, M., Premerlani, W., Eddy, F., & Lorensen, W. (1991). Object-Oriented Modeling and Design. Prentice Hall.

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

Spring. (2026). Spring Boot Reference Documentation. Versión estable consultada durante la revisión: 4.1.1.

Universidad Autónoma de Yucatán. Programa de Ingeniería de Software I. Unidad VI: Técnica de Modelado de Objetos (TMO).

Documento guía proporcionado por el usuario: Primer parcial de Sistemas de Información II, usado como referencia para la organización de flujos PUDS y artefactos de análisis, diseño, implementación y pruebas.

8. ANEXOS PREVISTOS

- Qr u url
-